ZPDPatch: Supervising Program Repair for Programming Assignments with Students' Own Execution Evidence

***Abstract*—We present ZPDPATCH, a trajectory-supervised repair technique that returns a program passing the tests while keeping as much of the student's own submission as it can. Automated Program Repair has substantially raised the repair rate for buggy student submissions. However, existing techniques frequently return a program that replaces most of what the student wrote, a return that no longer continues the student's own attempt, because the target that supervises them is an accepted solution settled without reference to what the student has already tried. An average return of a state-of-the-art retrieval-based technique changes 2.5 times as much of the program as an average student's own step, and more than the largest step the median student ever took unaided. Supervising repair on a student's own trajectory is challenging because a system that reads that trajectory also improves by drawing more candidates, and because a new target rule changes the training population with it. ZPDPATCH mines supervision pairs from execution evidence, trains one adapter per target rule, and freezes a portfolio by executed validation repairs, returning the submission unchanged when no candidate passes. Experimental results on 997 online-judge inputs from problems seen in training and 250 from held-out problems show that within an edit budget of 20 AST operations ZPDPATCH repairs 44.0%, 3.2 times as many as the retrieval-based technique (13.9%), and that its Progress rule keeps 5.6–6.5 points more of the submission's tokens and 8.9–10.4 points more of its lines at a cost of 11.2–19.2 points of unrestricted coverage. The same tradeoff reproduces on 181 student-held-out Introductory Programming (CS1) Java trajectories, where the Progress rule keeps 4.8–5.1 points more of the submission.**

***Index Terms*—programming-exercise feedback, automated program repair, submission trajectories, target construction, CS1**

I. INTRODUCTION

Automated Program Repair (APR) [1]–[4] has been widely studied for programming assignments as a way to give every rejected submission feedback that a course cannot write by hand [5]. Given a buggy submission and the problem's test suite, a technique returns a patch, that is, a program that passes every test, so the student receives a working version of their own assignment [6], [7]. To this end, a technique decides which part of the submission to change, synthesizes a replacement from the material it is given, and returns only a candidate that the tests accept. A student submits repeatedly to the same assignment, so the technique faces a sequence rather than one buggy program [8].

The key design decision is what that material is, that is, what the repair is supervised toward. Existing techniques for programming assignments fall into three groups by where they take it from. Alignment-based techniques align the submission with other students' passing programs or an instructor's solution [9]–[14]; language-model-based techniques generate a patch from the buggy program and the problem statement [6], [7], [15]–[19]; and retrieval-based techniques look up a learner-specific solution to steer that generation [20]. In addition, next-step hint generators draw one edit from a solution space built out of past correct programs [21]–[24]. These techniques have raised the repair rate for student submissions considerably, and put a next step within reach of more students than a tutor can see [5], [25].

However, existing techniques do not know how far a student can get with guidance. Where a return aims is settled without reference to which tests the student currently passes. Feedback works better when it is specific to the learner's own work [26], [27], and it helps only as far as the student can follow the change [28], [29]. The Zone of Proximal Development divides a learner's ability into what they can do unaided, what they can do with guidance, and what stays out of reach, and support has to land in the middle region [30]. A submission is what the student produced unaided, so it marks where that first region ends, and feedback chosen without regard to the student's level can land anywhere beyond it. The distance between a return and the submission is observable [31]–[33] (Section II-B). An average return of the most recent technique, LSGen [20], is 2.5 times an average student step. Consequently, existing techniques hand back somebody else's finished solution rather than the next step the student could take. In this work, we define *trajectory-conditioned repair* as repair whose supervision depends on the sequence leading to the current submission.

In this paper, we present ZPDPATCH, a new trajectory-supervised technique that repairs a buggy submission toward the next step the student could reach with guidance. Our key idea is that the student's own trajectory already records that step, since the submission that actually came next is a program the student reached unaided soon after. This target cannot be swapped in on its own (Section II-B), so ZPDPATCH holds the sources, the example counts and the candidate budget fixed and adds a matched-peer control. To this end, ZPDPATCH operates in three stages, Mine, Train, and Select. The Mine stage pairs a buggy submission, or a retained prefix, with a later submission under three rules. Progress takes the next state that improves the verdict or passes more tests, Strict the next better-verdict state, and Answer the accepted endpoint. Unlike

retrieval-based techniques that mine the same pairs to look up a solution, these pairs supply the supervision signal itself. The Train stage trains an independent adapter per rule. The Select stage freezes a triple of checkpoints by their executed validation repairs rather than by training loss. As a result, what a student receives is a complete program, so how much of their own submission it replaces can be measured and bounded.

Experimental results demonstrate that ZPDPATCH changes the least of a student's program among the techniques we compare, and under a bounded edit repairs more submissions than LSGen. We evaluated it on 1,247 Python submissions from 382 CodeNet problems [34] and 181 Java submissions from 17 Introductory Programming (CS1) exercises [8], against LSGen [20] and the same base model used zero-shot. In terms of repair rate under a declared edit budget, ZPDPATCH fixed 44.0% of the Python submissions at 20 Abstract Syntax Tree (AST) operations against LSGen's 13.9%, and 14.7 points more on average across six budgets. Regarding patch size, its patches changed 21.05 AST nodes on average, less than a quarter of LSGen's 88.27. Under matched sources, Progress targets kept 5.6 and 6.5 percentage points more of the student's tokens on the inputs both rules fixed, although they fixed 19.2 and 11.2 points fewer. We further show the same two directions on student-disjoint CS1 Java, where Answer targets lead on 15 of 17 exercises, and that three fixed candidate draws alone account for 15.1 points of the repair rate. Unrestricted, however, no configuration of ours exceeded LSGen's 80.2%, so we retain the current-only Answer triple and report the Progress rule as the option for when locality binds (Section IV).

Contributions. We summarize our contributions below:

- **Observation.** We show that solution-supervised repair seldom returns a step from where a student stands, and that within a step of 20 AST operations a trajectory target repairs 3.2 times as many submissions (Section II-B), motivating repair that takes its target from the student's own trajectory.
- **New technique.** We present ZPDPATCH, a trajectory-supervised repair technique for programming assignments. Our core technical contribution is an algorithm that pairs each buggy submission with a later submission of its own trajectory under execution-evidence rules, trains one adapter per rule, and freezes a checkpoint triple by executed validation repairs, with the sources, the example counts and the candidate budget held fixed.
- **Evaluation.** We demonstrate the effectiveness of ZPDPATCH by comparing it with LSGen, a zero-shot model, and controls on the target's source, prompt, decoder and candidate budget: it changes the least of the student's program, while no configuration of ours leads unrestricted. We supply the paired outcomes, condition manifests, metric code, and a claim-to-evidence map as anonymous supplementary material.

II. PRELIMINARIES

A. Automated Program Repair for Programming Assignments

Existing techniques for repairing programming assignments aim to turn a buggy submission into a program that passes every test of its problem. A problem p comes with a problem context d_p, its statement and resource limits, and a test suite $\mathcal{E}$. That aim asks only whether a return passes the tests, and nothing about what the student submitted. We define the rest of the setting on the student's submissions themselves. A student working on p submits repeatedly. We define a submission as a triple $s = (c, v, o)$, where c represents its source code, v denotes the verdict the platform recorded, and o indicates execution evidence that sends each test of $\mathcal{E}$ to its re-executed outcome. The recorded verdict and the replayed outcome stay in separate fields, because runtime variability can change what is observed. The *trajectory* $\tau = [s_1, \ldots, s_n, s_{ac}]$ orders those states chronologically, with $s_1, \ldots, s_n$ buggy and s_{ac} the first accepted submission, which serves only as evaluation ground truth and never reaches a technique. The submission to be repaired is the latest buggy one, and we call it the *buggy program* $b = s_n$. Cutting each τ at its first acceptance yields the *canonical* dataset and its frozen split.

Algorithm 1 APR for Programming Assignments

Input: Buggy program (b), Context (x), Ordered generators (S), Tests ($\mathcal{E}$), Edit budget (B)
Output: Returned program (f)

```
1:  procedure REPAIR(b, x, S, E, B)
2:      f ← b; q* ← EXECUTE(b, E)
3:      for g ∈ S do
4:          z ← Generate(g, x)
5:          if B < ∞ and ted†(b, z) > B then
6:              continue
7:          q ← EXECUTE(z, E)
8:          if q = 1 then
9:              return z
10:         if (q, −ted†(b, z)) > (q*, −ted†(b, f)) then
11:             f ← z; q* ← q
12:     return f
```

Algorithm 1 describes a generic repair procedure for programming assignments in the style of existing techniques [9]–[11]. It takes five inputs, a buggy program b, a context x, an ordered set S of generators, a test suite $\mathcal{E}$, and an edit budget B, and returns one program f. Line 2 makes the buggy program b the fallback for the return f and uses EXECUTE to record in q^* the fraction q of tests that b passes. Lines 3–4 take each generator g of the ordered set S in turn, and Generate builds one complete candidate z from the context x. Lines 5–6 skip a candidate whose $\mathrm{ted}^\dagger$ from b exceeds the edit budget B. Here $\mathrm{ted}^\dagger$ is AST edit distance when both programs parse, zero for unchanged code, and infinity otherwise. An unrestricted B therefore admits syntax repair. Line 7 executes the candidate and lines 8–9 return it as soon as every test passes. Lines 10–11 replace f with a candidate that passes a larger fraction, or the same fraction from closer to b, and line 12 returns f when no candidate passes. For now, let us set aside what the context x holds and where S and B come from; all three are fixed in Section III.

Existing techniques instantiate Algorithm 1 in different ways. Alignment-based techniques replace the generator at line 4 with an alignment against reference programs. CLARA [9] uses the expressions of a reference cluster sharing the control flow of b, Sarfgen [10] aligns same-control-flow references on their ASTs, Refactory [11] refactors references toward the control flow of b, and assignment-aware techniques vary how references are chosen [12]–[14]. Language-model-based techniques [6], [7], [15]–[19] replace line 4 and often the generator set at line 3 with a model. Retrieval-based LSGen [20] keeps that model and retrieves a learner-specific solution into the context x. Next-step hint generators [21]–[24] return one edit rather than a program, so lines 5 and 7 do not apply. In every case the generator is steered toward an accepted solution or a solution space, never toward the later submissions of the student being helped.

B. Limitation of Solution-Supervised Repair

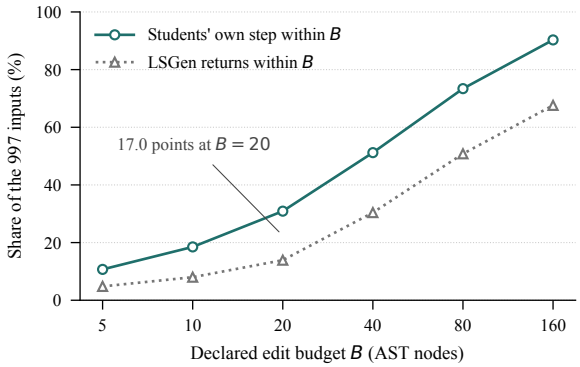


Fig. 1. Students' own steps and LSGen's returns within each budget

Our key observation is that solution-supervised repair does not return a step from where the student stands. A return that replaces most of b is not a continuation of it. Locating a student's regions takes measurement of its own (Section I), so we count how much of b the return keeps. We measure it by the $\mathrm{ted}^{\dagger}$ of Algorithm 1 and by token retention (Section IV). More precisely, we call a return whose distance from b is at most a declared budget B a *bounded step*, where B is the amount of a student's program a course is willing to give up to one repair.

Figure 1 measures how large a step the students themselves take and what LSGen repairs within it, over the 997 Seen inputs. On Seen the 2,581 steps with a defined distance have a median of 15, and one step leaves a median of 96.1% of the submission's tokens in place. Unrestricted, the returns of LSGen sit at 88.27 and those of the zero-shot model at 27.25. We call the largest step a student took unaided on a problem their *demonstrated reach*. Its median is 39, with quartiles at 15 and 83. An average LSGen return is 2.5 times an average student step and larger than the biggest step the median student took alone. Twenty AST operations is close to the median student step and within the reach of 30.9% of students, where LSGen repairs 13.9%, and 160 covers the reach of 90.3% of students. Surprisingly, four out of five of the submissions LSGen can repair are not repaired within one step, since it repairs 80.2% of them when nothing bounds the step. This measurement counts what a student changes at once, not which steps lay inside that student's region.

Prior techniques already ask for a small change: Refactory synthesizes minimal repairs [11] and LSGen instructs the model to make minimal changes [20]. One alternative is to bound the step of an existing technique. The 13.9% above is that alternative, and it costs LSGen 66.3 of its 80.2 points, because a generator steered toward an accepted solution has no smaller candidate to fall back on. Returning a hint sidesteps the budget, but the student still has to write the step, and the hint is still fitted to a solution space [22]–[24]. Another alternative is to swap the target for the student's own later submission and leave everything else alone. That is not a one-variable intervention: the same trajectories yield 40,454 Answer examples but 16,973 and 21,416 under Strict and Progress, and the source frequencies move too. What a rule retains moves with it too: among three wrong-answer submissions with pass vectors $(1,0,0)$, $(1,1,0)$, and $(0,1,1)$, verdict-only filtering cannot tell the second from the third, while test-wise progress retains the second, which keeps every passed test and gains one, and not the third, which gains one but loses another. Extra candidates also raise the repair rate on their own: three fixed draws alone account for 15.1 points (Section IV), so a comparison that does not hold the candidate budget fixed credits the target with them [7], [17], [19], [20], [25].

These limitations motivate our approach, which repairs under the guidance of the student's own trajectory. The goal of this paper is to repair a buggy program into one that passes the tests while keeping as much of the submission as possible. In terms of Algorithm 1, we train the generator toward the student's own later submission rather than toward an accepted solution or a solution space. A later submission is a program the student actually produced, so it is reachable by construction; we use it as a preference and let the tests decide what is returned. Our work thus tackles two challenges, (1) that the student's later submission marks the next step only coarsely and may itself be accepted, and (2) that attributing a gain to the target requires holding the sources, the example counts, and the candidate budget fixed.

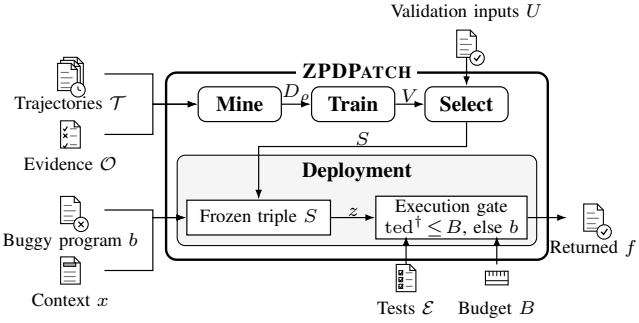


Fig. 2. Overview of ZPDPATCH

(a) Buggy submission b

```
1 N = int(input())
2 for i in range(N):
3     if i**2 > N:
4         print((i-1)**2)
5         exit()
```

(b) Progress target: the next submission

```
1 N = int(input())
2 for i in range(N):
3     if i**2 >= N:
4         print((i-1)**2)
5         exit()
```

(c) Answer target: the accepted submission

```
1 n = int(input())
2 ans = 0
3 for a in range(10**5 + 1):
4     if a**2 <= n:
5         ans = max(ans, a**2)
6 print(ans)
```

Fig. 3. Running example (p03556), blank lines omitted

III. Our Approach

A. Overview

Algorithm 2 ZPDPatch

Input: Trajectories ($\mathcal{T}$), Execution evidence ($\mathcal{O}$), Validation inputs (U), Seeds (Σ), Portfolio size (η_k), Edit budget (B)
Output: Frozen ordered checkpoint portfolio (S)

1: **procedure** ZPDPatch($\mathcal{T}, \mathcal{O}, U, \Sigma, \eta_k, B$)
   /* Stage 1. Mine */
2: $(D_A, D_S, D_P) \leftarrow (\emptyset, \emptyset, \emptyset)$
3: **for** $\tau = [s_1, \ldots, s_n] \in \mathcal{T}$ **do**
4: $D_A \leftarrow D_A \cup \mathsf{TerminalPairs}(\tau)$ ▷ Answer
5: **for** $\rho \in \{S, P\}$ **do**
6: $R \leftarrow [s_1]$ ▷ retained prefix
7: **for** $s_t \in [s_2, \ldots, s_n]$ **do**
8: **if** $\mathsf{Improves}_\rho(R[-1], s_t; \mathcal{O})$ **then**
9: $D_\rho \leftarrow D_\rho \cup \{(R, c_t)\}$; $R \leftarrow R \,\|\, [s_t]$
   /* Stage 2. Train */
10: $V \leftarrow \emptyset$
11: **for** $(\rho, \sigma) \in \{A, S, P\} \times \Sigma$ **do**
12: $V \leftarrow V \cup \{\mathsf{Train}(D_\rho, \sigma)\}$
   /* Stage 3. Select */
13: **for** $g \in V$ **do**
14: $R_g^B \leftarrow \mathsf{Validate}(g, U, B)$ ▷ $R_g^B \subseteq U$
15: $S \leftarrow \mathsf{Select}(\{R_g^B\}_{g \in V}, \eta_k)$ ▷ $\eta_k = 3$
16: **return** $\mathsf{FreezeAndOrder}(S)$

Figure 2 illustrates how ZPDPatch repairs a buggy program under the guidance of the student's own trajectory. ZPDPatch takes the inputs of Algorithm 1, namely the buggy program b, the context x, and the test suite $\mathcal{E}$, together with the trajectories $\mathcal{T}$ of a training corpus and their execution evidence $\mathcal{O}$. It processes these inputs in the three stages of Algorithm 2, namely Mine, Train, and Select. The Mine stage pairs each buggy submission of a $\tau \in \mathcal{T}$, or a retained prefix of it, with a later submission under three target rules, yielding one dataset per rule (Section III-C). The Train stage trains one adapter per rule and seed, giving a pool of checkpoints (Section III-D). The Select stage runs each checkpoint on validation inputs and freezes the ordered triple whose executed repairs cover the most of them (Section III-E). At deployment the frozen checkpoints are the generator set S of Algorithm 1, which returns the first candidate that passes every test and otherwise returns the submission unchanged.

ZPDPatch differs from existing techniques in where it obtains the target and in what it holds fixed around it. First, the target is the student's own later submission. Existing techniques supervise the generator toward an accepted solution [9]–[11], a solution space built from past correct programs [21]–[23], or a retrieved learner-specific solution [20], so what a return aims at is settled without the student's own evidence; the target of ZPDPatch is the next state of the same trajectory that execution evidence shows to be an improvement. Second, because swapping the target also moves the training population and the candidate budget (Section II-B), ZPDPatch holds the sources, the example counts, and the candidate budget fixed across rules and admits a matched-peer target as a control, so a difference in outcome can be read as the target's. The three stages run once per corpus, and deployment issues at most one call per selected checkpoint, so the deployed budget is three calls.

B. ZPDPatch

Algorithm 2 details how ZPDPatch instantiates Algorithm 1. Taking $\mathcal{T}$ and $\mathcal{O}$ in addition to the inputs of Algorithm 1, it builds the generator set offline and requires no instructor solution, no pool of reference programs, and no learner model.

At lines 2–9, Mine scans each trajectory τ and emits the pairs of each rule (Section III-C). The Answer rule pairs every non-accepted state with the accepted endpoint (line 4), whereas the Strict and Progress rules carry a retained prefix R and append a state only when the rule's improvement predicate holds on the execution evidence $\mathcal{O}$ (lines 7–9), emitting (R, c_t) before the state is appended. At lines 10–12, Train fits one adapter per rule and seed, so the pool V holds $|\Sigma|$ checkpoints per rule (Section III-D). At lines 13–16, Select runs each checkpoint on the validation inputs U under the edit budget B, records the inputs it repairs, and freezes the ordered triple whose union covers the most of U, where $\eta_k = 3$ in our experiments (Section III-E). Choosing by executed validation repairs rather than by training loss ties the choice to the only outcome a deployment can observe. The returned portfolio S is what line 3 of Algorithm 1 iterates over at deployment.

The order of S affects which passing program a student receives, not whether one is found, because every checkpoint of S is invoked until a candidate passes and the gate returns b unchanged when none does. Hence, given deterministic test outcomes, the repair set of the portfolio contains that of each of its members.

C. Mine

The goal of the Mine stage is to make every supervision pair traceable to an observed source and a later submission (lines 2–9). Figure 3 shows what that choice changes on one trajectory: the Progress target (b) is one AST operation from (a), while the Answer target (c) is 33 operations away and takes another approach. Submissions are grouped by problem and user, chronologically ordered, and cut at the

first acceptance. Canonical normalization removes consecutive duplicates while preserving the source code used for generation and execution. A whole-trajectory filter instead excludes a trajectory before splitting if any required prompt–target configuration exceeds 4,096 tokens. Exclusion changes the evaluated population, but never truncates a program or history.

The coarse severity function assigns 0 to Accepted (AC), 1 to Wrong Answer (WA), 2 to Time or Memory Limit Exceeded (TLE/MLE), 3 to Runtime Error (RE), and 4 to Compile Error (CE); unrecognized and internal outcomes occupy the most severe category. Strict compares recorded verdicts. Progress additionally compares replayed outcomes when the recorded verdict strings agree. Its cache must be complete, and compared vectors must have equal, nonempty testcase keys.

For equal testcase universes, $o_t \prec_\times o_r$ holds when every test in o_t has severity no greater than its counterpart in o_r, with a strict decrease on at least one. Formally, the improvement predicates are:

$$\begin{aligned} \mathsf{Improves}_S(r,t) &\iff \mathrm{sev}(v_t) < \mathrm{sev}(v_r), \\ \mathsf{Improves}_P(r,t) &\iff \mathsf{Improves}_S(r,t) \\ &\quad \vee \ (v_t = v_r \wedge o_t \prec_\times o_r). \end{aligned} \tag{1}$$

Intuitively, Equation (1) retains coarse improvements and progress hidden by an unchanged verdict. A severity improvement such as RE to WA need not increase the pass fraction, and a verdict-improving edge does not require testwise dominance.

Answer emits an independent pair for each non-AC source and the accepted endpoint. Thus $[s_1, \ldots, s_9]$ yields $([s_1], c_9), \ldots, ([s_8], c_9)$; it does not expose accumulated prefixes to Answer training. Strict and Progress start with $R = [s_1]$, compare each later state with the *last retained* state, and append it only when the predicate holds. Before appending s_t, the builder emits (R, c_t). For WA, improved WA, AC, Strict emits $([s_1], c_3)$, whereas Progress emits $([s_1], c_2)$ and $([s_1, s_2], c_3)$. The accepted endpoint remains eligible in both rules. Although Strict's pairwise predicate is contained in Progress's, their independently scanned edge sets need not be nested.

Chronological indices strictly increase, so construction terminates without cycles. With fixed-key evidence, retained edges also decrease the appropriate verdict/test severity rank. These are builder invariants, not evidence of a learned ZPD effect. Binary accepted/non-accepted mining retains 14,027 of 16,973 Strict pairs and 18,009 of 21,416 Progress pairs. Models retrained on them agree with the canonical ones on 84.6% and 87.5% of Seen repair decisions (Section IV-E), so we read the ranking as operational rather than universal.

D. Train

The goal of the Train stage is to learn each relation independently while keeping model and optimization comparable (lines 10–12). The implementation fine-tunes Qwen2.5-Coder-7B-Instruct with 4-bit Quantized Low-Rank Adaptation (QLoRA) [35], [36]. Adapters have rank 16, scaling 32, and dropout 0.05, targeting attention and feed-forward projections. Canonical training uses one epoch, learning rate 2×10^{-4}, and seeds 2027–2029. Assistant target tokens supply cross-entropy loss. The role-neutral prompt contains the statement, limits, permitted submissions, execution/edit summaries, and a request for a complete solution. It names neither the rule nor a preferred patch size, so specialization must come from the data rather than explicit instructions.

Targets are future submissions available offline; inference inputs contain only observed states. Current-only controls reset positions and remove earlier entries, preventing length leakage through position labels. Answer training is already single-source, so supplying history at inference creates a distribution change. The history-removal retraining control instead holds targets fixed and changes earlier-submission serialization. Source-matched target training holds current input and count fixed.

The paired-target control inner-joins records by problem, user, submission identifier, and the exact buggy program, retaining sources with distinct Answer and Progress targets. Its 7,389 training and 931 validation examples per condition use identical current-only inputs. The separate peer control requires another user on the same problem and in the same split, with exactly equal target execution evidence. Writing y_u for the same-user target and y_v for its peer replacement, for semantic-token diff distance d the discrepancy $\delta = |d(c, y_v) - d(c, y_u)|$ must satisfy $\delta \le 2$ or $\delta / \max(1, d(c, y_u)) \le 0.1$. Scarce evidence groups are visited first; ties prioritize distance discrepancy, reuse, and identifier, with donor reuse capped at three. This greedy rule yields 5,231 training and 306 validation examples per condition. Equal outcomes and nearby distances do not equate structure or style, so its measured effect belongs to the entire replacement rule, not isolated provenance.

E. Select

The goal of the Select stage is to deploy a bounded portfolio chosen without final-test outcomes (lines 13–16). For validation inputs U, let R_v^B be inputs repaired by checkpoint v within edit budget B. Formally, selection maximizes:

$$f_B(S) = \frac{1}{|U|} \left| \bigcup_{v \in S} R_v^B \right|, \qquad |S| = 3. \tag{2}$$

Intuitively, Equation (2) rewards complementary successes, not low training loss or the presence of every relation. Validation uses one fixed-hash trajectory from each of 461 eligible problems. The mixed pool has nine checkpoints and 84 triples; requiring one checkpoint per relation instead permits 27. The primary unconstrained mixed choice is Answer2027, Answer2028, and Progress2027. The relation-constrained choice is a different condition. Answer-9Choose3 receives nine Answer checkpoints and the same 84-triple search. Ties use recorded pass-rate, improvement, and deterministic identifier priorities. Budget-indexed selection repeats the declared objective at $B \in \{5, 10, 20, 40, 80, 160\}$, without choosing a budget from test results.

TABLE I
CORPUS AFTER WHOLE-TRAJECTORY FILTERING

Group	Part.	Probl.	Tests	Traj.	Subm.	Prefixes	Users
Seen	Train	492	9,446	14,027	54,481	40,454	4,012
	Valid	492	9,446	1,754	6,719	4,965	1,248
	Test	492	9,446	1,754	6,598	4,844	1,250
Unseen	Test	54	642	259	954	695	236
Total		546	10,088	17,794	68,752	50,958	–

Selected checkpoints and order are frozen. Each emits at most one independent candidate, and the first observed acceptance stops generation. If none passes all tests, Algorithm 1 prioritizes pass fraction and then smaller distance. Because unchanged code is feasible, for the same fixed execution measurements the returned pass fraction cannot decrease, and a finite edit budget cannot be exceeded. This is not per-test non-regression: equal fractions can conceal gains and losses on different tests. It also does not guarantee non-regression under fresh execution, an unseen suite, or resource variability.

Terminal pairing is linear in trajectory length; replaying n submissions on m tests requires $O(nm)$ executions and evidence storage. The 84-triple search is bounded at our pool size, not a scalable optimizer.

IV. EXPERIMENTS

In this section, we evaluate ZPDPATCH to answer the following research questions:

- RQ1: How effectively does ZPDPATCH repair buggy submissions?
- RQ2: How does the target rule change what a repair keeps of the student's program?
- RQ3: What do candidate breadth, checkpoint identity, and serialized history contribute?
- RQ4: Which findings replicate across problems, prefixes, model scales, and a classroom corpus?

We implemented ZPDPATCH on top of Qwen2.5-Coder-7B-Instruct [35], an open code model of a scale a course can host. All experiments were conducted on a single machine equipped with one NVIDIA RTX 5090.

A. Experimental Settings

Baselines. Zero-shot uses the same base model without fine-tuning and appends verdict/pass-count feedback after failure. LSGen retains repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval [20]; its same-problem Seen-Train references are unavailable under our Unseen information constraint, so that condition is not evaluated. Answer-1 uses one Answer checkpoint. Answer-3Seed uses seeds 2027–2029 on the same 40,454 Answer pairs. Answer-9Choose3 adds seeds 2030–2035 and selects three from nine. Mixed-9Choose3 denotes the primary unconstrained ZPDPATCH pool.

Benchmarks. Table I describes the Project CodeNet Python800 corpus [34] by partition (Part.) and submissions (Subm.); its Seen problems and tests are shared across partitions, so the user counts are not additive. We retain trajectories with at least three submissions, a non-AC state before the first acceptance, and an accepted endpoint in the benchmark reference set. Whole-trajectory 4,096-token filtering removes 3,660 trajectories before splitting. Problems ranked by eligible trajectory count supply the top 90% to Seen (492) and the remaining 54 to Unseen; Seen trajectories are split 80:10:10 within every problem. The primary endpoint is the last recorded non-AC submission of an eventually successful trajectory, and runtime eligibility requires the buggy program to remain non-AC and the recorded accepted endpoint to pass the available tests on replay. This leaves 997 of 1,754 Seen test trajectories, from 328 problems, and 250 of 259 Unseen trajectories, from all 54 problems; excluded trajectories do not enter the denominator. CodeNet users are online-judge participants, not a verified classroom population. We also use TIKTOC's [8] release of authentic Java CodeWorkout submissions and tests from an Introductory Programming (CS1) course: 1,125 trajectories from 224 students and 17 exercises, with 181 test trajectories from 33 held-out students. CodeWorkout is a practice platform where a course assigns short method-level exercises with instructor-authored tests. A separate exercise split has 10/3/4 train/validation/test exercises and 605/216/304 trajectories.

TABLE II
PRIMARY PYTHON EVALUATION. H: HISTORY INPUT; C: CURRENT-ONLY

Method	Input	Seen		Unseen	
		RR (%)	TED	RR (%)	TED
Zero-shot	H	30.7	27.25	62.0	17.46
LSGen	H	80.2	88.27	–	–
Answer-1	C	58.1	30.27	69.6	15.30
Answer-3Seed	H	61.7	24.52	73.2	13.38
Answer-9Choose3	H	59.8	25.46	74.8	15.19
Mixed-9Choose3	H	61.5	21.05	74.8	11.40
Answer-3Seed	C	72.0	37.94	81.2	16.01
Answer-9Choose3	C	69.5	35.41	82.0	20.06

Evaluation Metrics. We measure three kinds of outcome: whether the returned program passes the tests, how large its change is, and how much of the submission it keeps. Repair Rate (RR) is the fraction of eligible inputs whose returned program passes every test. Tree Edit Distance (TED) between ASTs measures current-to-returned change [37], [38] and is undefined when either program does not parse. Undefined cases remain in unrestricted RR but not in TED summaries, and finite gates reject undefined-distance changes. Unpaired TED averages successful, parseable returns, while paired locality uses joint successes. Retention divides tokens or nonempty stripped lines matched by `SequenceMatcher` by the source count. Unless stated otherwise, brackets give 95% percentile bootstrap intervals over problem clusters [39], [40], Java resamples students and exercises separately, and equal-problem estimates average problem means. McNemar tests are

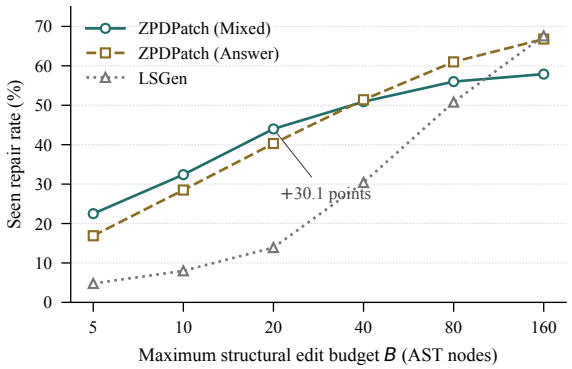


Fig. 4. Seen repair rate at each declared edit budget

supplementary to those intervals [41], and Holm adjustment covers the history-removal retraining tests [42].

B. RQ1: Repair Effectiveness

TABLE III
SOURCE- AND COUNT-MATCHED TARGET REPLACEMENT

Population	RR (%)		Retention (pp)	
	Answer	Progress	Tokens	Lines
Seen (997)	53.2	34.0	+5.6	+8.9
Unseen (250)	76.8	65.6	+6.5	+10.4
Java (181)	85.6	50.8	+4.8	+5.1

RQ1 asks how effectively ZPDPATCH repairs buggy submissions. To this end, we compared every configuration and baseline on the same inputs, both unrestricted and under a declared edit budget. In terms of unrestricted repair rate, Table II identifies current-only Answer-3Seed as the retained deployment configuration within our implementation, at 72.0% Seen and 81.2% Unseen. Relative to current-only Answer-1, its paired gain is 13.9 points [11.71, 16.29] and 11.6 points [7.32, 16.23]. RQ3 separates checkpoint identity from opportunity count. Current-only Answer-9Choose3 trades 25 Seen repairs for two Unseen ones (205/250 against 693/997), so we keep the fixed triple. Current-only mixed-pool reselection chooses exactly that triple.

Operational locality. Figure 4 gives the repair rate at each declared budget over the 997 Seen inputs. At TED 20, Mixed repairs 44.0% of Seen inputs versus LSGen's 13.9%, a 30.1-point difference [26.45, 33.82]. Across the six prespecified budgets its mean advantage over LSGen is 14.7 points [12.03, 17.43]. The advantage is budget-indexed: LSGen leads again at 160. On Unseen the same order holds through 20 and reverses at 40.

The history-input mixed portfolio repairs 613/997 Seen and 187/250 Unseen inputs. Its Seen gain over Zero-shot is 30.8 points [26.43, 35.09], or 26.1 [21.24, 30.93] with equal problem weights. Mixed-only and Zero-shot-only successes are 342 against 35 (McNemar $p < 10^{-63}$). This is not a history-specific effect: Mixed nearly ties Answer-3Seed, while its matched nine-checkpoint contrast is +1.7 points [-0.79, 4.29] on Seen and 0.0 [-3.70, 3.75] on Unseen.

LSGen repairs 800/997 Seen inputs versus Mixed's 613/997, a difference of -18.8 points [-23.04, -14.46]; 80 versus 267 exclusive successes give $p < 10^{-23}$. On joint repairs, Mixed's TED is lower by 43.34 [38.47, 48.44] relative to LSGen and 8.46 [5.05, 12.55] relative to Zero-shot. Conditioning excludes unilateral successes, so smaller patches do not cancel the deficit. The cause of that deficit is the search space: LSGen retrieves a learner-specific solution for every input and retrieves again after each failure, whereas our deployment issues three calls from frozen checkpoints. The deficit sits at the unrestricted end, and the order reverses once an edit budget is declared. Overall, the deployed configuration repairs the most within a declared budget and trails LSGen only when nothing bounds the edit, so we retain current-only Answer-3Seed.

C. RQ2: Target Construction and Locality

RQ2 asks how the target rule changes what a repair keeps of the student's program. To this end, we replaced the target while holding the sources and the example counts fixed, and repeated the comparison on student-held-out Java.

Matched targets. Table III changes the target while holding current inputs and training counts fixed. Progress-minus-Answer RR is -19.2 points [-22.96, -15.54] on Seen and -11.2 [-17.37, -4.88] on Unseen, with exact McNemar $p = 7.85 \times 10^{-34}$ and $p = 6.17 \times 10^{-5}$. The Answer dataset's advantage is therefore not only more training records. The intervention is the target rule, not a clean incorrect-versus-correct contrast. Under the matched inputs the Progress rule repairs 34.0% of Seen inputs and improves 43.4%, so a further 9.4 points receive a program that advances without passing, against 53.2% and 65.0% for the Answer rule. The same two gaps are 6.4 and 7.6 points on Unseen, and 9.4 against 2.2 on Java, where Answer targets already repair 85.6% outright. Part of the coverage the Progress rule gives up therefore returns as partial progress, counted by test execution and not by comprehension. The cause lies in the target itself: a Progress target is not required to pass, so the generator trained on it returns programs that advance without passing.

Progress's locality benefit is visible on joint successes. Among 299 Seen and 154 Unseen jointly repaired inputs, 278 and 134 have defined source-to-repair TED for both methods. Answer-minus-Progress mean TED is 10.08 [7.53, 12.99] and 6.83 [4.58, 9.39]. Token retention improves by 5.6 [4.29, 6.94] and 6.5 [4.26, 8.95] points; line retention improves by 8.9 [6.64, 11.16] and 10.4 [6.95, 14.25]. The 21 and 20 undefined-TED cases remain in RR and are not declared unrepairable. These measures describe successful outputs that retain more of the submission, not easier comprehension.

Java retraining uses 600 training and 118 validation pairs per target. Answer targets repair 155 trajectories and Progress targets 92, a difference of 34.8 points; Progress-minus-Answer intervals are [-44.91, -25.47] under student clustering and [-47.31, -23.79] under exercise clustering. Among 90 joint

TABLE IV
JAVA MATCHED-TARGET OUTCOMES OVER 17 EXERCISES AND 90 JOINTLY REPAIRED STUDENTS

Repairs (17 exercises)		Retention (90 students)	
Answer higher	15	Answer higher	5.6%
Tied	2	Tied	63.3%
Progress higher	0	Progress higher	31.1%
Extra Answer repairs	63	Median gain	9.6
Largest gap	12	Retention range	-21.1 / 49.7

(a) Progress repair: 20 of 21 lines kept (TED 1)

```
1 N=int(input())
2 S=input()
:
12   while j>=0:
:
21 print(ans)
```

(b) Answer repair: 3 of 21 lines kept (TED 83)

```
1 N=int(input())
2 S=input()
3 cnt=S.count('R')
4 ans=S[:cnt].count('W')
5 print(ans)
```

Fig. 5. What each rule returns for one jointly repaired submission

repairs, Progress targets increase lexical-token retention by 4.8 points and line retention by 5.1 points, with intervals [2.79, 7.19] and [2.92, 7.25]. This is evidence for a coverage–locality tradeoff across two languages and populations.

What a student receives. Cohort means do not describe an individual student, so Table IV resolves the Java contrast by exercise and by student. One exercise accounts for 12 of the 63 additional Answer repairs, and there Progress targets repair 3 of 15 trajectories against 15. Most students who receive a working repair from either rule get back the same amount of their own code, about a third get more, by a median of 9.6 points, and a few get less. Line retention behaves the same, zero for 64.4% of students with maximum 47.6. Figure 5 illustrates that tail on Python. The student's own preceding edit changed `i<j` to `i<=j` and left 93.3% of the tests passing; the Progress-target repair (a) changes one line more, `j>0` to `j>=0`, whereas the Answer-target repair (b) returns a five-line program that keeps three of the student's 21 lines. Both pass every test under the same gate and budget.

Peer targets. The separate outcome- and distance-matched replacement raises RR from 50.4% to 54.7% on Seen and from 78.4% to 84.4% on Unseen. Same-minus-peer intervals are [-7.71, -0.84] and [-11.79, -0.41], with exact McNemar $p = 0.005$ and $p = 0.032$. Equal outcomes with nearby distances still permit differences in structure, so the contrast does not isolate user identity. Matched peer targets improve coverage here; one's own trajectory is not uniquely necessary under this control. Overall, target construction moves coverage and locality in opposite directions under matched inputs and in the same direction on student-held-out Java, and the effect is not uniform across exercises or students.

D. RQ3: Component Efficacy

RQ3 asks what candidate breadth, checkpoint identity, and serialized history each contribute. To this end, we compared controls that change one of them at a time within the stored executions.

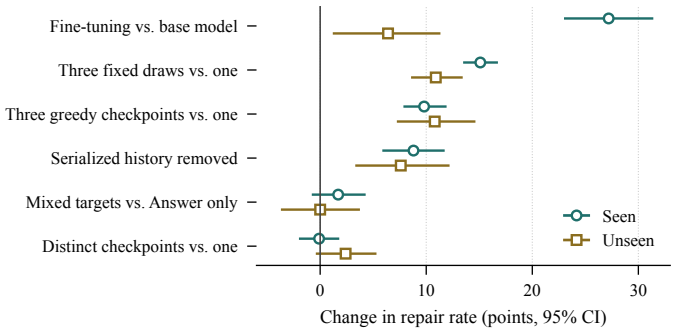


Fig. 6. What each change does to the repair rate

Fixed candidate breadth. Figure 6 reports what each matched control does to the repair rate, with 95% intervals. For three stored draws with success indicators a_{ij}, the expected uniformly selected one-draw success for input i is $\frac{1}{3}\sum_j a_{ij}$, whereas the three-draw union succeeds when $\max_j a_{ij} = 1$. Their paired difference isolates additional opportunities within the identical candidate set. Replacing greedy decoding by one stochastic draw costs 7.6 points [-9.81, -5.41], whereas execution-selecting three of those identical fixed draws contributes 15.1 points [13.48, 16.76] on Seen. It shows why comparing a portfolio only with one adapter overstates the evidence for its training signal.

The temperature sweep retains all declared conditions. Through $T = 1.0$ the one-draw rate stays within 41.1–43.1% on Seen, the three-draw union reaches 60.0/75.2%, and at $T = 1.5$ the decoder collapses to 15.8/35.2%. Fine-tuning also matters: the matched base-model three-draw union reaches only 30.4/65.6%, so adaptation adds 27.2 points [22.99, 31.41] on Seen and 6.4 points [1.19, 11.34] on Unseen. Breadth alone therefore does not explain all improvement over the base model.

At matched temperature, replacing three draws of one checkpoint by one draw from each of three checkpoints changes RR by -0.1 points [-2.00, 1.80] on Seen and 2.4 points [-0.41, 5.31] on Unseen. These intervals do not establish a consistent advantage from checkpoint diversity. A deterministic multi-seed gain can coexist with this result because sampling from one checkpoint already supplies diversity.

A current-only factorial on one inference stack tests this explanation. Three greedy checkpoints gain 9.8 [7.85, 11.92] and 10.8 [7.23, 14.64] points over one, but under the matched stochastic decoder distinct checkpoints add only 2.1 points [0.00, 4.16] and 1.2 points [-2.93, 5.93], with the equal-problem Seen interval including zero. Because the backend changes the absolute rates, our recommendation covers the stack we evaluated.

Serialized history. Removing earlier submissions from the frozen triples raises repair for both pools. The gains are 8.8/11.1 Seen points and 7.6/6.0 Unseen points for Mixed/Answer, with all intervals excluding zero. Matched retraining without that context shows no significant difference

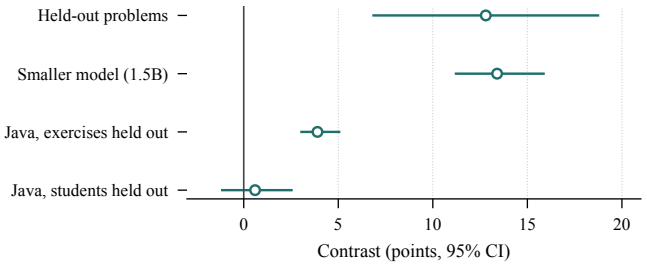


Fig. 7. Where the findings replicate

after Holm correction, with smallest adjusted $p = 0.313$, so serialized history is not part of the measured advantage and the deployed configuration uses current-only prompts.

Answer-9Choose3 receives the same nine-checkpoint pool size and 84-subset opportunity as Mixed. The primary five-fold cross-fitted mixed-minus-Answer contrast is +2.2 points [-0.19, 4.63], compared with the non-cross-fitted +1.7 [-0.79, 4.29]. Overall, candidate breadth and adaptation each contribute measured gains, whereas distinct checkpoints add no consistent advantage once the stochastic decoder is matched and serialized history is not supported as a necessary input.

E. RQ4: Generality

RQ4 asks which findings replicate across problems, prefixes, model scales, and a classroom corpus. To this end, we measured the same contrasts again on held-out problems, earlier prefixes, a smaller model, and the classroom corpus.

Problems and execution suites. Figure 7 collects the contrast and its interval in each replication setting. Within Unseen, Mixed exceeds Zero-shot by 12.8 points [6.83, 18.75], with 45 versus 13 exclusive successes and exact McNemar $p = 3.01 \times 10^{-5}$. On the 122 joint repairs with defined TED it reduces TED by 6.41 [3.43, 9.89]. The higher absolute Unseen RR is not evidence of stronger generalization: the partitions differ in availability and composition. Matching all 54 Unseen problems to distinct Seen problems on six pre-repair difficulty covariates changes the raw gap into -4.6 equal-problem points [-14.18, 5.30]. We therefore rely on within-split paired contrasts rather than the raw cross-split difference.

A second check splits the authored suite into an observed slice and a held-back slice. Among the inputs each repairs on the observed slice, 189/192 (98.4%) and 184/186 (98.9%) also pass the held-back slice, and 13 versus 8 exclusive joint successes yield $p = 0.383$. Repeating the procedure on all 328 Seen problems with a distinct seed yields 97.8% and 97.9% confirmation. These are held-back slices of the same authored tests, so the conditional percentage must not replace the full-denominator repair rate.

Earlier prefixes. Across 2,770 Seen and 674 Unseen executable prefixes of eventually successful trajectories, current-only Answer-3Seed reaches 69.6% and 80.1% RR. The early, middle, and last thirds reach 70.0%, 67.6%, and 71.4% on Seen and 82.9%, 77.3%, and 81.8% on Unseen. Multiple prefixes from one trajectory are dependent, so this rate is not added to the 997-input result.

Model scale and classroom data. At 1.5B parameters three checkpoints still gain 13.4 points over one [11.16, 15.92], but mixed-minus-Answer is -0.8 [-2.62, 1.07]. Mixed reaches 50.3% and the nine-checkpoint Answer control 51.1% on Seen. This smaller model reproduces the utility of multiple opportunities, not an unrestricted mixed-target advantage.

The Java portfolio results differ from RQ2's direct matched-target retraining. Its interval is [2.99, 5.11] on held-out exercises and [-1.20, 2.59] on held-out students. Absolute RR is 56.2% versus 52.3% over 304 exercise-held-out trajectories and 84.0% versus 83.4% over 181 student-held-out trajectories. The exercise estimate rests on four clusters: refitting with one exercise left out keeps the contrast between 3.6 and 4.4 points, and single exercises range from 2.5 to 5.5 points. Four clusters cannot support a broad classroom-population claim.

Representation and overlap. Binary accepted/non-accepted mining removes the fine severity ordering but also changes the target set. Retraining changes Progress RR by +2.3 Seen and -2.8 Unseen points in the canonical-minus-alternative direction. Because the changes go in opposite directions, the ordering is not a free implementation choice. For Mixed and Answer-9Choose3, exact matches between a returned repair and a training-time accepted program occur in 0.6% and 1.3% of selections, near-duplicates above a 0.90 token-similarity threshold in 3.8% and 5.3%, and median maximum similarity to a same-problem training target is 66.7% and 68.6%. Low exact-copy rates do not exclude structural reuse or pretraining contamination [43]. Overall, the matched-target tradeoff replicates on student-held-out Java and candidate breadth remains useful at a smaller model scale, whereas mixed-target unrestricted superiority does not replicate consistently.

F. Discussion

Implications for instructors. Our results suggest three ways a course can use these repairs. First, declaring an edit budget still leaves repairs to show: within 20 AST operations our configuration repairs 44.0% of the Seen inputs where LSGen repairs 13.9% (Section IV-B). Second, which rule a course deploys follows from whether coverage or locality binds, since the Progress rule keeps more of the student's tokens on the inputs both rules fix and repairs fewer of them (Section IV-C). Third, that choice is better made per exercise than for a whole course, because cost and gain are uneven across exercises and students (Section IV-C).

Delivering repairs. A returned program can be shown as it is, reduced to the lines it changed, or passed to a hint generator as input [15], [24]. Since handing a model a solution steers its hints toward that solution, a return that keeps more of the student's own approach is likely the better input for such hints as well.

G. Threats to Validity

(1) Runtime variability, undefined source ASTs, and Python whitespace affect labels and distances. We retain unknowns,

denominators, and whitespace-sensitive corrections, but hashes establish provenance, not correctness, and source and peer matching do not equate all target semantics or styles. (2) Acceptance is not proof of semantic correctness [44], same-suite slices are not independent specifications, and TED and retention do not measure cognitive load or subsequent learning. CodeNet does not identify novices or measure mastery, Java supplies classroom submissions rather than an intervention observing feedback use, and the per-exercise and per-student resolution describes which students would receive a different repair, not which students benefit, with Figure 5 one post-hoc case that carries no estimate. We display a CodeNet program because the course data terms do not permit redistributing student submissions. (3) The comparison covers the retrieval-based and model-based lines, not the alignment-based one. Filtering favors short, eventually successful Python trajectories and permits Seen user overlap, Java adds one course context and only four held-out exercises, and cluster bootstrap and equal-problem estimates do not eliminate small-cluster uncertainty, multiplicity, or the bias of analyses chosen after earlier results. These distinctions limit educational and target-quality interpretations.

V. Related Work

APR for Programming Assignments. Generate-and-validate repair spans search, learned transformations, and language models [1]–[4]; search analyses [45] and execution-based evaluation [46], [47] separate candidate quality from exposure, and training signals range from neural patches and ensembles to synthetic corruption and self-supervision [48]–[51], including none at all [52], [53]. Beyond the techniques described in Section II, many techniques repair student submissions. Among alignment-based techniques, clustering, alignment, and refactoring exploit correct programs of the same problem [9]–[11], assignment-aware selection varies which references are used [12]–[14], and tutoring systems propagate feedback and generate hints [21]–[23], [54]. Among language-model-based techniques, models support repair, syntax feedback, conversations, peers, and counterexamples [6], [7], [15]–[19]. We compare against the retrieval-based and language-model-based lines under one information constraint and exclude the others for a reason each: alignment-based techniques align against accepted programs of the same problem, an input our Unseen condition withholds; assignment-aware selection varies that same input; and next-step hint generators return one edit rather than a program, so the gate and the budget of Algorithm 1 do not apply. We do not introduce portfolio repair; we isolate trajectory targets with same-target seeds, equal search spaces, fixed draws, and current-only controls.

Feedback on the Student's Own Work. A second line fits its guidance to a record of student work. Data-driven hints read a solution space or a past cohort's paths and project the submission onto it, so two students at the same position receive the same next step whatever tests their own code passes [22]–[24]; a hint and a program are also measured on different scales [5], [25], [55]. LSGen also mines chronological wrong-to-later-correct submission pairs for retrieval [20], so trajectories are not our novelty; it does not separate the history's contribution from that of extra candidates, however, which is why we hold both fixed together. ZPD and scaffolding concern supported performance, not program edit distance [30]–[33], knowledge tracing models proficiency from interactions [56]–[58], and TIKTOC incorporates testcase evidence [8]. Next-step hints, feedback ladders, and novice-response studies examine guidance more directly [24], [59], [60], while feedback theory emphasizes its interpretation and use [26]–[29]. In contrast, ZPDPatch reads the submission's own execution evidence and returns a complete program whose target is a step the same student later took, so it exposes a coverage–locality tradeoff rather than dominance over feedback strategies. To our knowledge, ZPDPatch is the first repair technique that makes a step from the student's own trajectory the training target rather than a retrieval input. This paper does not claim that a learner's serialized history improves generated repairs: our deployed configuration prompts with the current submission alone, and the supplement reports the controls behind that choice, including their negative outcomes.

VI. Conclusion

Automated Program Repair has substantially raised the repair rate for buggy student submissions. However, existing techniques settle what a return aims at without reference to the student's own attempts, so a return can replace most of the submission. This paper aims to repair a buggy submission toward a step taken from the student's own trajectory. To this end, ZPDPatch mines supervision pairs from execution evidence, trains one adapter per rule, and freezes a portfolio by executed validation repairs. Experimental results demonstrate the same coverage–locality tradeoff in matched Python and student-held-out Java experiments, uneven across exercises and students, while candidate breadth and model adaptation account for the rest of the improvement. We therefore retain the current-only Answer triple and report the Progress rule as the option for when locality binds. We hope that ZPDPatch helps a course return repairs that continue a student's own attempt rather than replace it.

Data Availability

An anonymous supplement contains the paired outcomes, condition manifests, metric code, negative results and claim–evidence links; its offline verifier recomputes every reported contrast. Student source and model weights are excluded under the CodeWorkout/TIKTOC [8] and CodeNet [34] terms, and no student intervention was conducted.

Research process. OpenAI Codex assisted with design, code, orchestration, diagnostics, and manuscript preparation under author instructions, and automated agents executed authorized jobs; we do not claim authors manually ran every job. Outcomes come from retained execution records rather than generated prose. The experiment program stopped at its declared cutoff of 21 October 2026, and this synthesis adds no training, generation, or testcase rerun.

References

[1] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: A survey," *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2019.

[2] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–24, 2018.

[3] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the 31st International Conference on Software Engineering*, 2009, pp. 364–374.

[4] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, 2012.

[5] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Transactions on Computing Education (TOCE)*, vol. 19, no. 1, pp. 1–43, 2018.

[6] H. Joshi, J. C. Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, "Repair is nearly generation: Multilingual program repair with LLMs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5131–5140.

[7] J. Zhang, J. P. Cambronero, S. Gulwani, V. Le, R. Piskac, G. Soares, and G. Verbruggen, "Pydex: Repairing bugs in introductory python assignments using llms," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 1100–1124, 2024.

[8] Z. Duan, N. Fernandez, A. Hicks, and A. Lan, "Test case-informed knowledge tracing for open-ended coding tasks," in *Proceedings of the 15th International Learning Analytics and Knowledge Conference*, 2025, pp. 238–248.

[9] S. Gulwani, I. Radiček, and F. Zuleger, "Automated clustering and program repair for introductory programming assignments," *ACM SIGPLAN Notices*, vol. 53, no. 4, pp. 465–480, 2018.

[10] K. Wang, R. Singh, and Z. Su, "Search, align, and repair: data-driven feedback generation for introductory programming exercises," in *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*, 2018, pp. 481–495.

[11] Y. Hu, U. Z. Ahmed, S. Mechtaev, B. Leong, and A. Roychoudhury, "Refactoring based program repair applied to programming assignments," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 388–398.

[12] J. Heo, H. Jeong, D. Choi, and E. Lee, "Referent: Transformer-based feedback generation using assignment information for programming course," in *2023 IEEE/ACM 45th international conference on software engineering: Software engineering education and training (ICSE-SEET)*. IEEE, 2023, pp. 101–106.

[13] D. Choi, J. Heo, and E. Lee, "Automated feedback generation for multiple function programs," in *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2021, pp. 582–583.

[14] D. Choi and E. Lee, "Automated feedback generation for programming assignments through diversification," in *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 2025, pp. 230–241.

[15] T. Phung, J. Cambronero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares, "Generating high-precision feedback for programming syntax errors using large language models," in *Proceedings of the 16th International Conference on Educational Data Mining*, 2023, pp. 370–377.

[16] F. Liu, Z. Liu, Q. Zhao, J. Jiang, L. Zhang, Z. Sun, G. Li, Z. Li, and Y. Ma, "Fastfixer: An efficient and effective approach for repairing programming assignments," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 669–680.

[17] B. Yang, H. Tian, W. Pian, H. Yu, H. Wang, J. Klein, T. F. Bissyandé, and S. Jin, "Cref: An llm-based conversational software repair framework for programming tutors," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 882–894.

[18] Q. Zhao, L. Zhang, F. Liu, Y. Liu, Z. Yan, Z. Chen, Y. Zhou, J. Jiang, G. Li, Z. Sun *et al.*, "Peer-aided repairer: Empowering large language models to repair advanced student assignments," *Empirical Software Engineering*, vol. 31, no. 2, p. 33, 2026.

[19] P. Orvalho, M. Janota, and V. M. Manquinho, "Counterexample guided program repair using zero-shot learning and maxsat-based fault localization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 649–657.

[20] Z. Dai, Z. Zhao, H. Wang, X. Tang, S. Wu, C. Yao, Z. Gao, and J. Chen, "Learner-tailored program repair: A solution generator with iterative edit-driven retrieval enhancement," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 36, 2026, pp. 30 467–30 475.

[21] C. Piech, J. Huang, A. Nguyen, M. Phulsuksombati, M. Sahami, and L. Guibas, "Learning program embeddings to propagate feedback on student code," in *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 1093–1102.

[22] K. Rivers and K. R. Koedinger, "Data-driven hint generation in vast solution spaces: a self-improving python programming tutor," *International Journal of Artificial Intelligence in Education*, vol. 27, no. 1, pp. 37–64, 2017.

[23] T. W. Price, Y. Dong, and D. Lipovac, "iSnap: towards intelligent tutoring in novice programming environments," in *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*, 2017, pp. 483–488.

[24] L. Roest, H. Keuning, and J. Jeuring, "Next-step hint generation for introductory programming using large language models," in *Proceedings of the 26th Australasian computing education conference*, 2024, pp. 144–153.

[25] C. Koutcheme, N. Dainese, S. Sarsa, J. Leinonen, A. Hellas, and P. Denny, "Benchmarking educational program repair," *arXiv preprint arXiv:2405.05347*, 2024.

[26] J. Hattie and H. Timperley, "The power of feedback," *Review of educational research*, vol. 77, no. 1, pp. 81–112, 2007.

[27] V. J. Shute, "Focus on formative feedback," *Review of educational research*, vol. 78, no. 1, pp. 153–189, 2008.

[28] S. Narciss, "Feedback strategies for interactive learning tasks," in *Handbook of research on educational communications and technology*. Routledge, 2008, pp. 125–143.

[29] V. Aleven and K. R. Koedinger, "Limitations of student control: Do students know when they need help?" in *International conference on intelligent tutoring systems*. Springer, 2000, pp. 292–303.

[30] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*, M. Cole, V. John-Steiner, S. Scribner, and E. Souberman, Eds. Cambridge, MA: Harvard University Press, 1978.

[31] T. Murray and I. Arroyo, "Toward measuring and maintaining the zone of proximal development in adaptive instructional systems," in *International conference on intelligent tutoring systems*. Springer, 2002, pp. 749–758.

[32] I.-A. Chounta, P. Albacete, P. Jordan, S. Katz, and B. M. McLaren, "The "grey area": A computational approach to model the zone of proximal development," in *European conference on technology enhanced learning*. Springer, 2017, pp. 3–16.

[33] N. Schulze Bernd and I.-A. Chounta, "Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure," in *International Conference on Artificial Intelligence in Education*. Springer, 2024, pp. 365–378.

[34] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker *et al.*, "Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks," *arXiv preprint arXiv:2105.12655*, 2021.

[35] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[36] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.

[37] K. Zhang and D. Shasha, "Simple fast algorithms for the editing distance between trees and related problems," *SIAM Journal on Computing*, vol. 18, no. 6, pp. 1245–1262, 1989.

[38] M. Pawlik and N. Augsten, "Tree edit distance: Robust and memory-efficient," *Information Systems*, vol. 56, pp. 157–173, 2016.

[39] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[40] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[41] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[42] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[43] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, pp. 143–153.

[44] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543.

[45] Y. Qi, X. Mao, Y. Lei, Z. Dai, and C. Wang, "The strength of random search on automated program repair," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 254–265.

[46] M. M. A. Haque, W. U. Ahmad, I. Lourentzou, and C. Brown, "Fixeval: Execution-based evaluation of program fixes for programming problems," in *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 2023, pp. 11–18.

[47] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[48] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 4, pp. 1–29, 2019.

[49] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "Coconut: Combining context-aware neural translation models using ensemble for program repair," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 101–114.

[50] M. Yasunaga and P. Liang, "Break-it-fix-it: Unsupervised learning for program repair," in *International conference on machine learning*. PMLR, 2021, pp. 11 941–11 952.

[51] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "SelfAPR: Self-supervised program repair with test execution diagnostics," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[52] C. S. Xia and L. Zhang, "Less training, more repairing please: revisiting automated program repair via zero-shot learning," in *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 959–971.

[53] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 1430–1442.

[54] E. L. Glassman, J. Scott, R. Singh, P. J. Guo, and R. C. Miller, "Overcode: Visualizing variation in student solutions to programming problems at scale," *ACM Transactions on Computer-Human Interaction*, vol. 22, no. 2, pp. 1–35, 2015.

[55] J. Yi, U. Z. Ahmed, A. Karkare, S. H. Tan, and A. Roychoudhury, "A feasibility study of using automated program repair for introductory programming assignments," in *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 2017, pp. 740–751.

[56] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User modeling and user-adapted interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[57] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," *Advances in neural information processing systems*, vol. 28, 2015.

[58] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-aware attentive knowledge tracing," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 2330–2339.

[59] H. Heickal and A. Lan, "Generating feedback-ladders for logical errors in programming using large language models," in *Proceedings of the 17th International Conference on Educational Data Mining*, 2024, pp. 947–951.

[60] R. Xiao, X. Hou, and J. Stamper, "Exploring how multiple levels of gpt-generated programming hints support or disappoint novices," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–10.